University of East Anglia Norwich
School of Computing Sciences

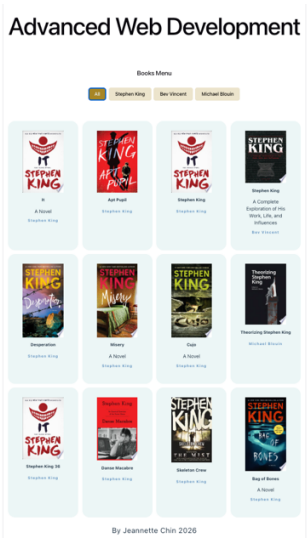
Advanced Web Development

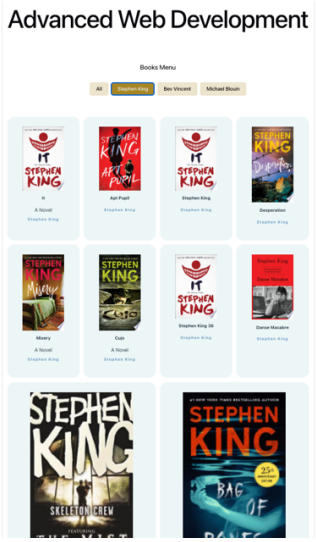
Excise 2.1: REACT Basic – Working with REACT States

The main learning objective is to understand REACT state and how to implement them in our application. To follow this exercise, you must complete the Exercise 2.1 (REACT Basics – Component, Props and JSX) first.

For this exercise, we will implement a filter menu, where user can select a particular author and the page will display their books accordingly.

Examples of final outputs:







Learning Objectives:

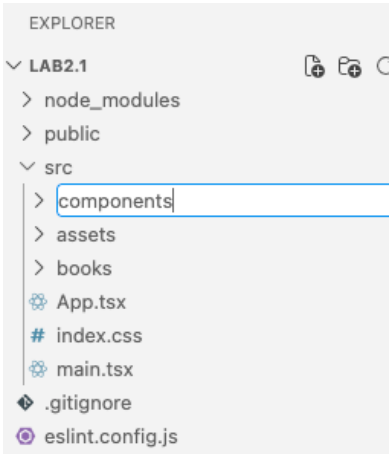
- Able to code REACT states.
- Able to implement filtering function using JavaScript and JSON data.
- Practice writing CSS rules to style the menu items.

We will implement the menu directly within the 'BookList' component, since this is where our book list is managed and rendered from the previous lab. The menu itself will consist of two parts: a 'MenuTitle' component and a 'MenuItems' component. Both of these new components will be organised inside a dedicated 'components' folder

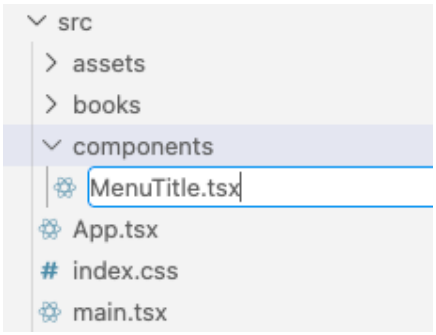
Note: for each lab exercise, I will start with a new folder and clone the code (from last lab exercise) that I committed to GitHub repo. I suggest you do the same.

Tasks:

1. Create a new folder called “lab2.1” as the root of this lab exercises. Clone last lab code (lab2) to this folder.
2. Bring up the development server (`npm run dev`), view the page on Google Chrome to make all is working.
3. Now create a new folder called ‘components’ inside the ‘src’ folder.



4. In this ‘components’ folder, create a new file called ‘MenuTitle.tsx’.



5. Code this ‘MenuTitle’ React functional component. This component takes a prop called ‘title’, which type is of string. The ‘title’ is then rendered inside a <h2> tag. Also, if title is not supplied, then a default value (of your choice) is rendered. The root/parent element is (i.e. <h2> should be nested inside element). Make sure the component is exported using ‘export default’. Below is my code for your reference:

src > components > MenuTitle.tsx > ...

```tsx
const defaultTitle = "Adv WebDev";

const MenuTitle = ({ title }: { title?: string }) => {
  return (
    <div>
      <h2>{title ? title : defaultTitle}</h2>
    </div>
  );
};

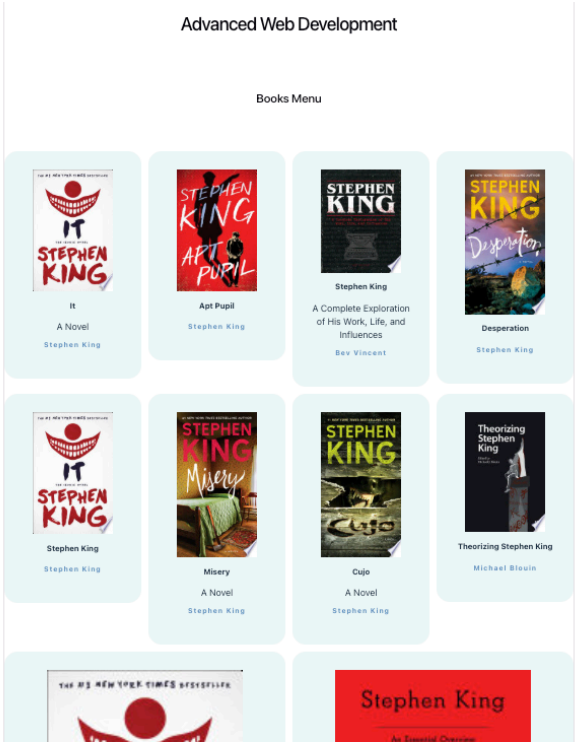
export default MenuTitle;

```

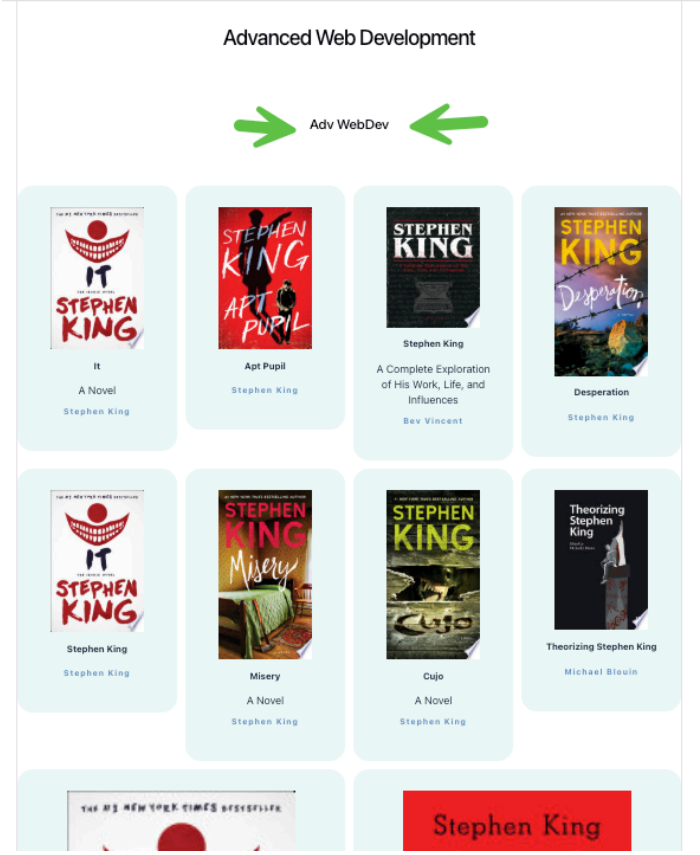
Notice that I have used ternary conditional operator to determine if a value for the title is supplied.

6. Next, we want to add this 'MenuTitle' component in our 'BookList' component. In 'BookList.tsx' file, do the following:
    a. Remove this line '<h2>BookList</h2>'.
    b. In its place, add a 'MenuTitle' component. Pass a prop to this component (i.e. title prop), with a value of your choice for your menu title. Make sure the 'MenuTitle' component is imported in this file. I use 'Books Menu' as my menu title, below is my output. Also test to see if without supplying a value will work.

       Supplying a prop:



No prop, a default value is rendered:



7. For our menu system, we want the user to be able to filter the content by author. In order to achieve this functionality, we will need two component states – one is governed the list of menu items (i.e. the authors list) and the other is governed the author who is selected (by the user). Let's go through this step by step:
    a. Import REACT 'useState' hook (line 4):

```
src > books > BookList.tsx >
1   import MenuTitle from "../components/MenuTitle";
2   import Book from "./Book";
3   import data from "./data.json";
4   import { useState } from "react";
5
```

    b. Create a REACT state using 'useState' hook (https://react.dev/reference/react/useState), this returns a state and a function. I will call the state returned as 'authors' and the function where we can use to modify the state as 'setAuthors'. By convention, the function name should begin with 'set' then follow by the name of the state being returned, using camelCase. The state must be declared inside the functional component otherwise it will not work. In our case, the component is 'BookList', so the 'useState' must be declared inside this component, see line 21.

```
19  const BookList = () => {
20    const title = "Books Menu";
21    const [authors, setAuthors] = useState();
22
```

c. The state we created in the above task is used to govern the menu items (of the authors), that we want to dynamically generate from the data. If you look at the JSON data, you will notice that there are entries with a same author (e.g. Stephen King). However, we do not want duplication in our menu items. So, the solution is to generate a list with unique author names. So far, we have already imported the JSON data in our 'BookList' component and unpacked the 'items' object from the data in our previous lab (see below), we can just use this to create our unique author list.

```
6   const { items } = data;
7
```

d. Follow the following steps:

   i. Create a const called 'tempAuthors', and grab the 'authors' objects from the 'items' object. We can use the 'map' function. Below is my code (see line 8 to line 10):

```
6   const { items } = data;
7
8   const tempAuthors = items.map((item) => {
9     return item.volumeInfo.authors;
10  });
11
```

   ii. Now print this 'tempAuthors' object on a console (Hint: use console.log), you will see below output:

```
(12) [Array(1), Array(1), Array(1), Array(1), Array(1), Array(1), Array(1), Arra
  0: ['Stephen King']
  1: ['Stephen King']
  2: ['Stephen King']
  3: ['Bev Vincent']
  4: ['Stephen King']
  5: ['Stephen King']
  6: ['Stephen King']
  7: ['Michael Blouin']
  8: ['Stephen King']
  9: ['Stephen King']
  10: ['Stephen King']
  11: ['Stephen King']
  length: 12
  [[Prototype]]: Array(0)
```

iii. From the output we can see that there are a lot of arrays there which is hard to work with. Here we can flatten them into one array using the '.flat()' method (see line 12)

```
6   const { items } = data;
7
8   const tempAuthors = items.map((item) => {
9     return item.volumeInfo.authors;
10  });
11
12  console.log(tempAuthors.flat());
13
```

iv. Save the file, and observe the console output:



v. As you can see, there are a lot of duplications in this list. Recall we want a list of menu items with unique values. The easiest way to achieve this is using JavaScript Set method (MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set). So, here is my code, see line 14:

```
6   const { items } = data;
7
8   const tempAuthors = items.map((item) => {
9     return item.volumeInfo.authors;
10  });
11
12  console.log(tempAuthors.flat());
13
14  const tempSet = new Set(tempAuthors.flat());
15
```

vi. Check out this 'tempSet' on the web console you will see the following output, the duplications are gone.



vii. Now we got our unique list of author names for the menu items for our users to select. However, we also want our user to be able to select 'All'. For this we can just create a new array and add this 'All' item to it as well. Here is my code, see line 15.

```
6   const { items } = data;
7
8   const tempAuthors = items.map((item) => {
9     return item.volumeInfo.authors;
10  });
11
12  console.log(tempAuthors.flat());
13  const tempSet = new Set(tempAuthors.flat());
14  console.log(tempSet);
15  const allAuthors = ["All", ...tempSet];
16
```

viii. Once you get to this point, we can carry on working with our REACT state in task (b) above.

e. Recall we have created a REACT state called 'authors', now we just set the initial state value using the unique name list we just developed, as follow, see line 19.

```
11
12  console.log(tempAuthors.flat());
13  const tempSet = new Set(tempAuthors.flat());
14  console.log(tempSet);
15  const allAuthors = ["All", ...tempSet];
16
17  const BookList = () => {
18    const title = "Books Menu";
19    const [authors, setAuthors] = useState(allAuthors);
20
```

f. Next, we want to render this list as menu items for our users, and also a function to allow our user to select a name (i.e. a menu item), the page will update the selected information accordingly. Let's create the function first, I called my function 'filterAuthor' (see line 21 to line 26), it takes the author's name (of type string) in the parameter, and checks it against the menu list and returns the selected one. Here is my code:

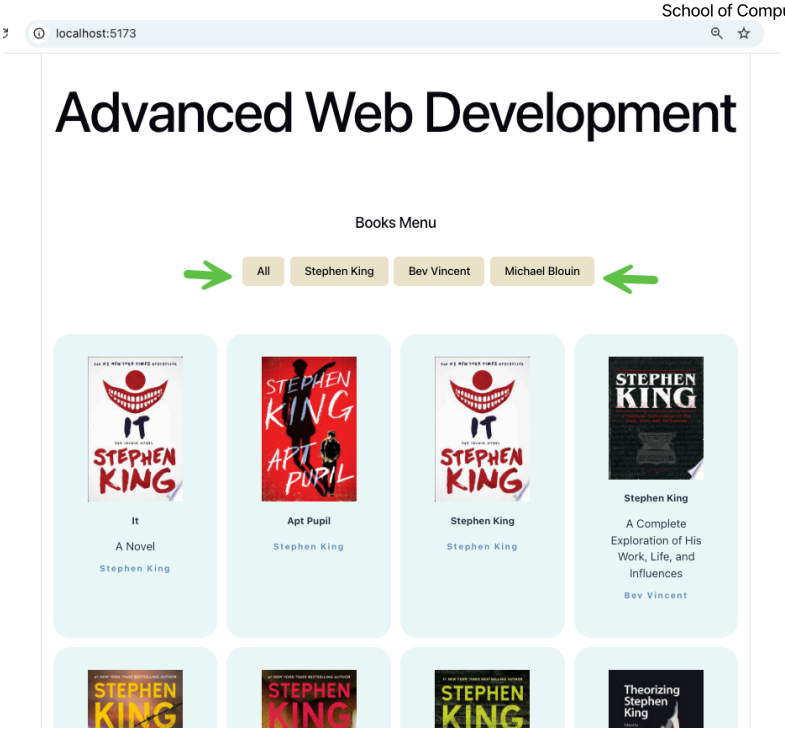
```
23  const filterAuthor = (author: string) => {
24    const newItems = allAuthors.filter((item) => {
25      return item === author;
26    });
27    console.log(newItems);
28  };
29
```

g. To render our menu items, simple iterate the authors array, and for each item found inside the array, render a button, use the item / author as the button label, and pass in the 'filterAuthor' callback function in onClick, as follow:

```
36          {authors.map((author) => {
37            return (
38              <button
39                key={author}
40                className="menu-btn"
41                onClick={() => {
42                  filterAuthor(author);
43                }}
44              >
45                {author}
46              </button>
47            );
48          })}
```

Note: In REACT, child element requires a unique key value.

h. Save the file. What's your output? Below is mine for your reference. You may create your CSS style rules to style your menu items!



i. If you click any of the menu button, it doesn't do anything, because we haven't implemented the handler yet. We will need a second state so to keep track of this selection. Here, my second REACT state is called 'author' (representing the selected one) and I will use 'All' as the initial state. Here is my code, line 20.

```
17  const BookList = () => {
18    const title = "Books Menu";
19    const [authors, setAuthors] = useState(allAuthors);
20    const [author, setAuthor] = useState(["All"]);
21
```

j. To trigger the rendering in REACT, we will need to call the 'setter' function, the one returned by 'useState' hook. In our case, we will call 'setAuthor' function, as this is the one being returned by setState().

```
17  const BookList = () => {
18    const title = "Books Menu";
19    const [authors, setAuthors] = useState(allAuthors);
20    const [author, setAuthor] = useState(["All"]);
21
```

k. The next question is where should we call this function? Logically we should call this function every time after a user selected a menu item/ name, so that it will trigger the re-rendering. So, the right place to call this function is inside the 'filterAuthor' function, here is my code, see line 27.

```
22  const filterAuthor = (author: string) => {
23    const newItems = allAuthors.filter((item) => {
24      return item === author;
25    });
26    console.log(newItems);
27    setAuthor(newItems);
28  };
29
```

l. We still need to implement the filtering functionality so that the correct rendering is performed. Here is my code, from line 53 to line 62:

```
52        <div className="booklist">
53          {items.map((item) => {
54            const { volumeInfo } = item;
55            if (author[0].length == 1 || author[0] == "All") {
56              return <Book {...volumeInfo} key={item.id} />;
57            } else {
58              if (volumeInfo.authors.includes(author[0])) {
59                return <Book {...volumeInfo} key={item.id} />;
60              }
61            }
62          })}
63        </div>
```

m. Save the file. With a bit of luck, your filtering system should work just like mine!

n. Once you get this working, create CSS rules to style the menu buttons, so that the items are displayed in the middle of the page, as shown in page 1.

Your next task is to push the code to CMP Stugit and name your repo as "<your-username>lab2.1"